

Київський національний університет
імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

Звіт

до лабораторної роботи №1
на тему:

**«Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 3,
варіант 8-6»**

Студента четвертого курсу
групи МІ-41
Мекшакова Артемія Олексійовича

Київ-2026

Зміст

1	Постановка задачі	1
2	Архітектура веб-клієнта	2
2.1	Маршрути і сесія	2
2.2	Спільні пакети і власні модулі	2
2.3	Реалізації інтерфейсів у двох клієнтах	3
2.4	Уточнення порівняно з UML етапу 1	3
3	Веб-клієнт	4
3.1	Вхід і реєстрація	4
3.2	Простір і таблиця файлів	4
3.3	Сортування за назвою	5
3.4	Фільтр за типом	5
3.5	Показ і приховування стовпців	7
3.6	Перегляд вмісту файлу	7
3.7	Завантаження кнопкою і перетягуванням	9
3.8	Скачування і видалення файлів	9
3.9	Синхронізація з локальною папкою	11
3.10	Конфлікт версій і його розв’язання	11
3.11	Автоматичне відстеження змін	12
3.12	Вузкий екран	13
3.13	Браузери без File System Access API	13
4	Публікація	14
4.1	Схема розгортання	14
4.2	Опублікована система	15
4.3	Сервер на Render	16
4.4	Веб-клієнт на Vercel	17
4.5	CORS і демонстраційні дані	17
4.6	Обмеження безкоштовних планів	17
5	Тестування	18
5.1	Unit-тести	18
5.2	Наскрізна перевірка	18
6	Інструкція із запуску	19
7	Висновки	19

1 Постановка задачі

Третій етап лабораторної роботи з курсу «Інформаційні технології» — веб-версія клієнта системи **типу 2**, тобто застосунку для роботи з віддаленою папкою з файлами на зразок спрощеного Google Drive, з графічним інтерфейсом у браузері та звітом. Веб-клієнт працює з тим самим сервером Shelf і його REST API, що й десктопний клієнт етапу 2.

Обидва бонуси виконано: завантаження перетягуванням є в обох клієнтах (у вебі — підрозділ

3.7, рис. 11 і 12), а сервер, база даних, сховище файлів і веб-клієнт опубліковані на хмарних сервісах із публічними адресами (розділ 4, рис. 19–21).

Індивідуальну частину задає варіант **8-6**:

Цифра	Список	Значення
8	ТИПИ файлів, вміст яких показується при кліку	.kt — як текст, .jpg — як зображення
6	ОПЕРАЦІЇ	сортування за назвою (зростання / спадання); фільтр «усі файли» / «лише .cpp» / «лише .png»

Веб-клієнт реалізує ту саму UML-модель етапу 1, що й десктоп, і використовує ті самі пакети `@shelf/shared` і `@shelf/ui` з тими самими іменами класів. Власного коду в ньому лише стільки, скільки вимагає браузер (розділ 2).

Адреси проєкту:

- сайт: <https://shelf-opal-two.vercel.app>;
- REST API: <https://shelf-api-1989.onrender.com>, документація за шляхом `/api/docs`;
- вихідний код: <https://github.com/NONAME-lf/shelf>.

2 Архітектура веб-клієнта

2.1 Маршрути і сесія

Веб-клієнт — пакет `@shelf/web` у каталозі `apps/web` на Next.js 16 з App Router і React 19, зі стилями Tailwind CSS 4 зі спільного `@shelf/ui/styles.css`. Маршрути — `/login`, `/register` і `/workspace`, а корінь `/` веде в простір або на вхід. Усі сторінки є клієнтськими компонентами (`'use client'`), тож збірка робить їх статичними файлами, а дані браузер отримує прямо від REST API. Сесію (JWT, користувача й адресу API) `SessionProvider` зберігає в `localStorage`, як і десктоп, і під час відкриття сайту перевіряє запитом `GET /api/auth/me`: відповідь 401, зокрема після 24 годин дії токена, веде на вхід, а недоступний сервер сесію не скидає. Адресу API задає під час збірки змінна `NEXT_PUBLIC_API_URL`, тому поля адреси сервера на екрані входу немає.

2.2 Спільні пакети і власні модулі

З `@shelf/shared` веб-клієнт без змін бере операції варіанта (`sortByName`, `filterByType`, `getVisibleFiles`), `toggleColumn`, `previewKindOf`, `FileApiClient`, `splitBySize`, `accountKey` і весь каталог `sync/` з `SyncEngine` і `computeStatus`, а з `@shelf/ui` — хук `useFileListController` і компоненти від `AuthForm` і `FileTableView` до `SyncPanel` і `ConflictDialog`, з яких складено екрани `LoginScreen` і `WorkspaceScreen`. Власні модулі веб-клієнта (сесію десктоп веде власним `session.ts`, решти в ньому немає; шляхи відносно `apps/web/src`):

Модуль	Роль
lib/session.ts, lib/SessionProvider.tsx lib/saveBlob.ts sync/fileSystemAccess.ts sync/browserLocalFolder.ts sync/indexedDbSnapshotStore.ts sync/folderBindingStore.ts	сесія в localStorage, перевірка токена, вихід після відповіді 401 збереження скачаного файлу через object URL і <a download> наявність showDirectoryPicker, перевірка і запит дозволу на папку BrowserLocalFolder — LocalFolder на File System Access API IndexedDbSnapshotStore — знімок стану кожної папки в IndexedDB FolderBindingStore — папка і прапорець відстеження облікового запису
sync/webSyncService.ts sync/useWebSync.ts sync/browserErrors.ts	WebSyncService — синхронізація, опитування кожні 5 с, запобіжники useWebSync — стан синхронізації і діалогу конфліктів для екрана explainBrowserError, explainPickerError — помилки браузера українською

2.3 Реалізації інтерфейсів у двох клієнтах

Інтерфейси зі специфікації спільні, різняться лише реалізації; докладно їх порівняє етап 4.

Інтерфейс або функція	Десктоп	Веб
LocalFolder	NodeLocalFolder (Node fs)	BrowserLocalFolder (File System Access API)
SnapshotStore	JsonSnapshotStore (JSON у userData)	IndexedDbSnapshotStore (IndexedDB)
прив'язка папки відстеження (UC14a)	SettingsStore (settings.json)	FolderBindingStore (IndexedDB)
скачування (UC11)	FolderWatcher на chokidar «Зберегти як», перетягування з вікна	опитування кожні 5 с у WebSyncService Blob і <a download> (saveBlob)

2.4 Уточнення порівняно з UML етапу 1

- **UC14a — опитування.** Браузер не повідомляє сторінку про зміни в папці, тому WebSyncService, поки ввімкнено прапорець і вкладка видима, кожні 5 с перечитує назви, розміри й час зміни файлів і синхронізує, лише коли цей список змінився.
- **Скачування через Blob.** Звичайне посилання не передає заголовок Authorization, тож FileApiClient.download отримує байти з токеном, а saveBlob їх зберігає.
- **LocalFolder.write повертає справжній час зміни.** Браузер не може задати файлу час зміни, і скачаний файл отримує поточний. SyncEngine записує в знімок саме час, який повернув write; з часом серверної версії наступна перевірка вважала б файл зміненим локально й вивантажила б його назад.
- **Прив'язка належить обліковому запису.** Handle папки і прапорець відстеження лежать в IndexedDB під ключем accountKey (користувач і адреса API), тож інший користувач того самого браузера їх не успадковує.
- **Файли .crswap пропускаються.** Це тимчасові файли, які Chrome створює поруч під час запису; BrowserLocalFolder не включає їх до списку.
- **Дозвіл після перезавантаження.** Handle папки переживає перезавантаження сторінки, а дозвіл Chrome забуває, і запитати його можна лише у відповідь на дію користувача: автоматичні запуски тому чекають, а «Синхронізувати» спершу запитує дозвіл.

- **Лише Chrome і Edge.** File System Access API мають тільки браузери на Chromium; у Firefox і Safari блок синхронізації показує пояснення (рис. 18).

3 Веб-клієнт

Компоновка вікна та сама, що в десктопі: ліворуч темна бічна панель, праворуч панель інструментів над таблицею, перегляд і конфлікти — у модальних вікнах. Знімки зроблено в Chromium у вікні 1280×800 на локальному стеку зі свіжими демонстраційними даними, під обліковим записом «Артем». Кожен користувач має власний простір, тож «Хто завантажив» і «Хто редагував» скрізь показують власника; різні імена були б лише за спільних папок, які поза обсягом проєкту.

3.1 Вхід і реєстрація

Екран входу (рис. 1) реалізує вимогу R1 і прецедент UC2: маршрут /login показує спільний компонент AuthForm з полями пошти й пароля. На відміну від десктопа, поля «Адреса сервера» тут немає, бо адресу API задано під час збірки. Після входу SessionProvider зберігає JWT, і сторінка переходить на /workspace. Якщо відповіді немає понад 5 с, під формою з'являється підказка «Сервер прокидається після простою — це може тривати до хвилини» (підрозділ 4.6). Кнопка «Вийти» завершує сесію (UC15).

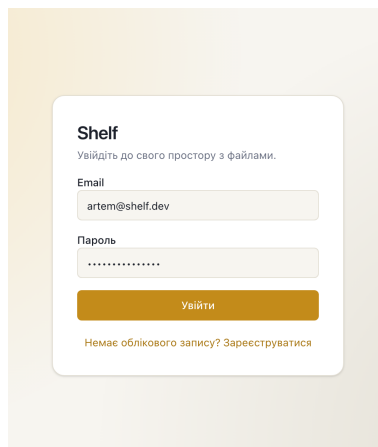


Рисунок 1. Екран входу веб-клієнта

Посилання під кнопкою відкриває маршрут /register (рис. 2, прецедент UC1) з додатковим полем «Ім'я», яке потім видно в бічній панелі і в стовпцях авторів. Сервер створює користувача разом із порожнім простором і відразу повертає токен, тож окремо входити не потрібно. Помилки, як-от уже зайнята адреса пошти (код 409), з'являються українською над кнопкою. На знімку форму заповнено для користувачки «Олена», але не надіслано.

3.2 Простір і таблиця файлів

Після входу відкривається простір користувача (рис. 3) — вимога R2, прецеденти UC3 і UC4. Таблиця містить десять демонстраційних файлів за зростанням назв: назву з піктограмою типу, розширення, розмір, дату й час створення і зміни, хто завантажив і хто редагував, кнопки скачування й видалення. Лічильник «10 з 10» порівнює видимі рядки з усіма файлами,

Shelf
Створіть обліковий запис, щоб отримати власний простір.

Ім'я
Олена

Email
olena@shelf.dev

Пароль

Зареєструватися

[Вже маєте обліковий запис? Увійти](#)

Рисунок 2. Форма реєстрації нового користувача

«Оновити» заново запитує список. Папку ще не прив'язано, тому блок «Синхронізація» показує «Не прив'язано» і кнопки «Обрати папку» та «Синхронізувати».

ПРОСТІР
Артем
artem@shelf.dev

СИНХРОНІЗАЦІЯ
Локальна папка
Не прив'язано
Обрати папку
Синхронізувати
Відстежувати зміни автоматично

СТОВПЦІ ТАБЛИЦІ
☒ Тип
☒ Розмір
☒ Дата створення
☒ Дата зміни
☒ Хто завантажив
☒ Хто редагував

Усі файли | Лише .cpp | Лише .png | 10 з 10 | Оновити | Завантажити

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ	ХТО ЗАВАНТАЖИВ	ХТО РЕДАГУВАВ	ДІЇ
backup.zip	.zip	576 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
class-diagram.png	.png	564 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
geometry.cpp	.cpp	226 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
Main.kt	.kt	319 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
matrix.cpp	.cpp	448 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
mountains.jpg	.jpg	30 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
README.md	.md	330 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
shelf-logo.png	.png	1,3 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
todo.txt	.txt	185 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
3bit.txt	.txt	148 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️

🔗 Вийти

Рисунок 3. Простір користувача: таблиця файлів з усіма атрибутами

3.3 Сортування за назвою

Сортування — перша операція варіанта (вимога R4, прецедент UC5). Клік по заголовку «Назва» перемикає напрямок, а піктограма поруч показує поточний. На рис. 4 файли впорядковано за спаданням: першим іде 3bit.txt, останнім — backup.zip. Працює та сама функція `sortByName` з `@shelf/shared`, що й у десктопі: `Intl.Collator` без урахування регістру і з природним порядком чисел. Мову порівняння бере браузер, тож в англomовному Chromium кириличні назви йдуть після латинських.

3.4 Фільтр за типом

Фільтр — друга операція варіанта (вимога R5, прецедент UC6): `TypeFilterControl` має кнопки «Усі файли», «Лише .cpp» і «Лише .png». На рис. 5 вибрано «Лише .cpp»: лишилися

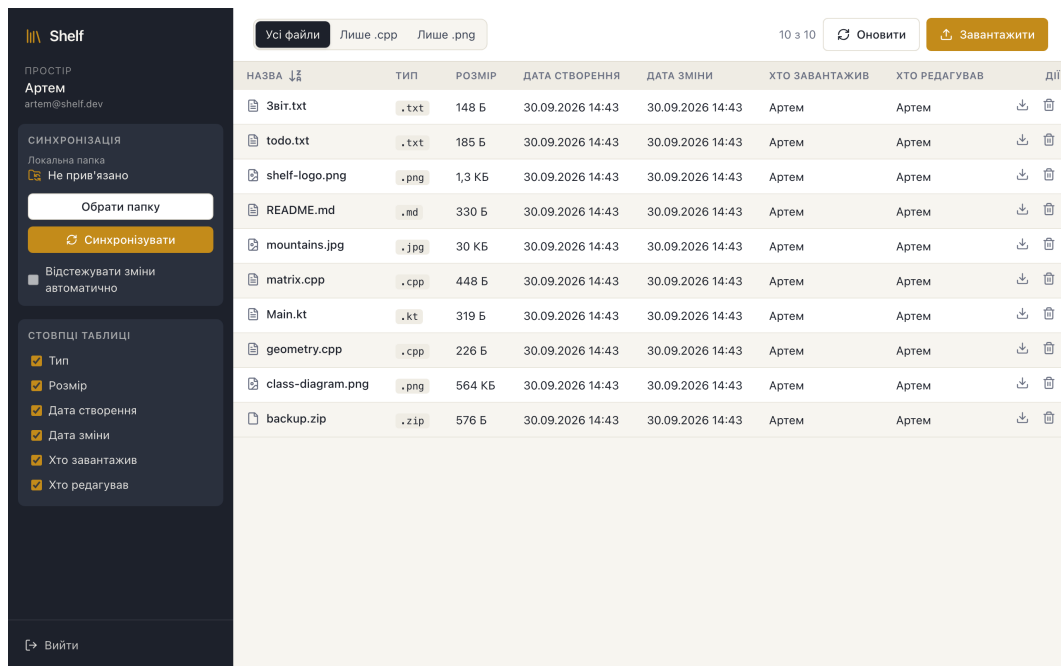


Рисунок 4. Файли, відсортовані за назвою за спаданням

geometry.cpp і matrix.cpp, а лічильник показує «2 з 10». Порівнюється саме розширення без урахування регістру, тож readme-cpp.txt фільтр не пройшов би. Фільтр накладається на відсортований список: `getVisibleFiles` обчислює `filterByType(sortByName(files, direction), filter)`, тож зміна фільтра не скидає обраного напрямку сортування.

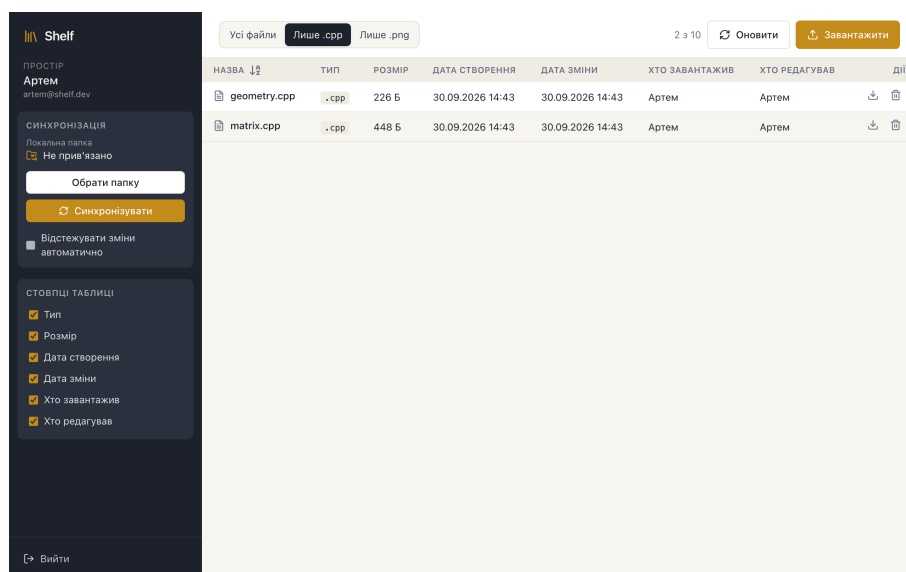


Рисунок 5. Фільтр «Лише .cpp»

На рис. 6 вибрано «Лише .png» — видно class-diagram.png і shelf-logo.png, лічильник знову показує «2 з 10», а «Усі файли» повертає повний список. Порядок відфільтрованих рядків задає сортування: наскрізна перевірка вмикає спадання, потім «Лише .cpp» і переконується, що matrix.cpp стоїть перед geometry.cpp. Обидві операції виконуються в браузері над уже отриманим списком, тому перемикання миттєве. Обов'язкові unit-тести цих функцій описано у звіті етапу 2.

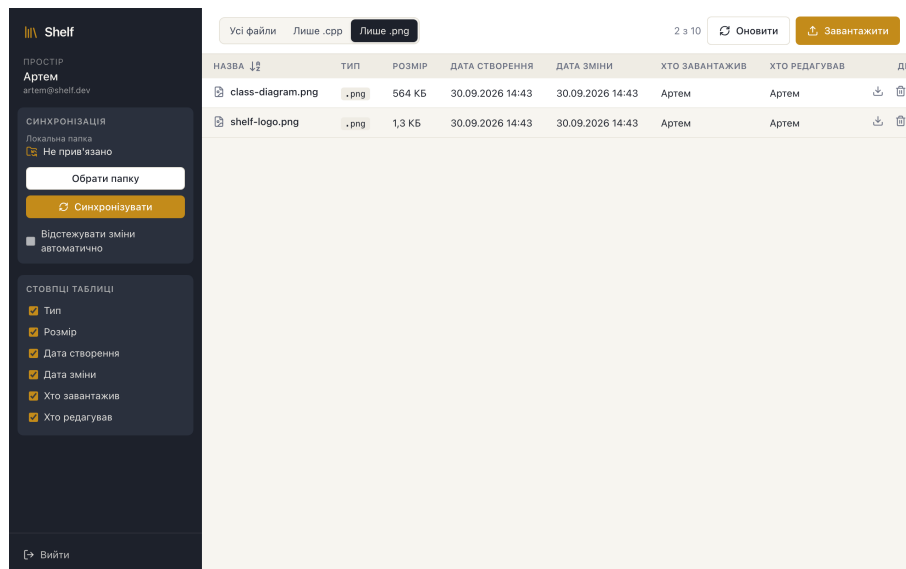


Рисунок 6. Фільтр «Лише .png»

3.5 Показ і приховування стовпців

Блок «Стовпці таблиці» (ColumnPicker) реалізує вимогу R3 і прецедент UC7: кожен стовпець, крім назви, має свій прапорець. На рис. 7 вимкнено «Розмір» і «Хто завантажив», і таблиця одразу втратила ці стовпці, а решта лишилася на місці. Назву приховати не можна: toggleColumn з @shelf/shared ігнорує таку спробу. Видимість стовпців зберігається в стані useFileListController і змінює лише відображення, тож список файлів заново не запитується.

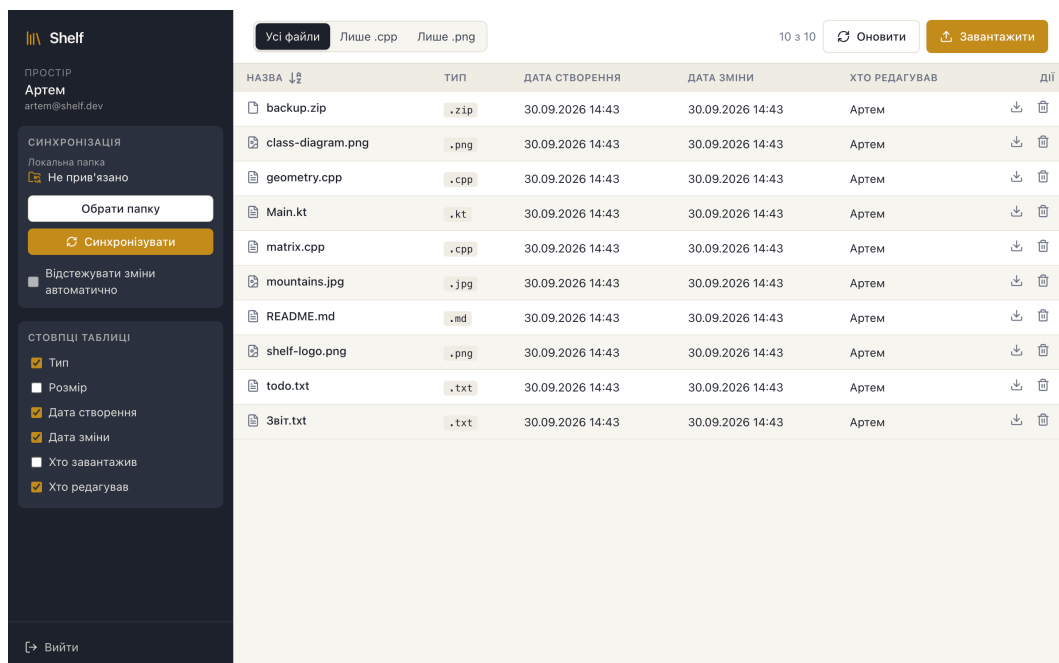


Рисунок 7. Приховано стовпці «Розмір» і «Хто завантажив»

3.6 Перегляд вмісту файлу

Клік по рядку відкриває модальне вікно PreviewDialog — вимога R6, прецедент UC8, тип варіанта. Спосіб перегляду визначає previewKindOf за розширенням, createPreview створює

TextPreview або ImagePreview, а вміст клієнт отримує через `FileApiClient.download` з токеном. На рис. 8 файл `Main.kt` показано як текст моноширинним шрифтом під розміром, датами й авторами. З великого текстового файлу показується перший мегабайт, а довгий вміст прокручується всередині вікна, тоді як заголовок і кнопки лишаються на місці.

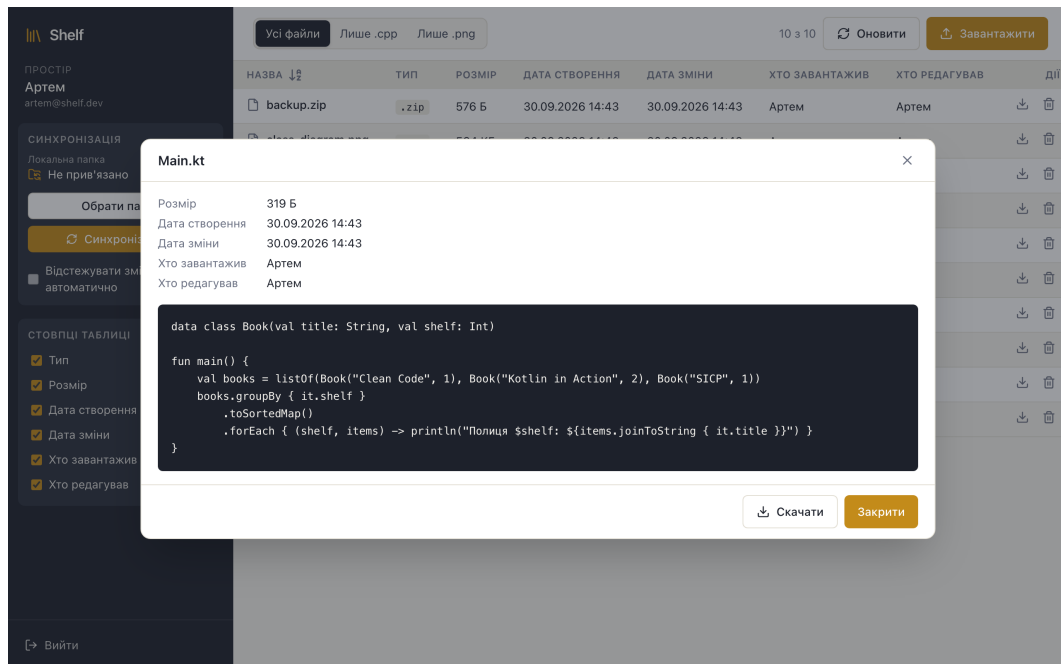


Рисунок 8. Перегляд файлу .kt як тексту

Файл `mountains.jpg` на рис. 9 відкрито як зображення: ImagePreview перетворює отримані байти на тимчасовий object URL, а картинка масштабується так, щоб вміститися у вікно разом з атрибутами. Типи перегляду варіанта не збігаються з типами фільтра, тому перегляд узагальнено: текстом показується будь-який текстовий файл, а картинкою — будь-яке растрове зображення, зокрема .png. Кнопка «Скачати» у вікні зберігає файл так само, як кнопка в рядку таблиці.

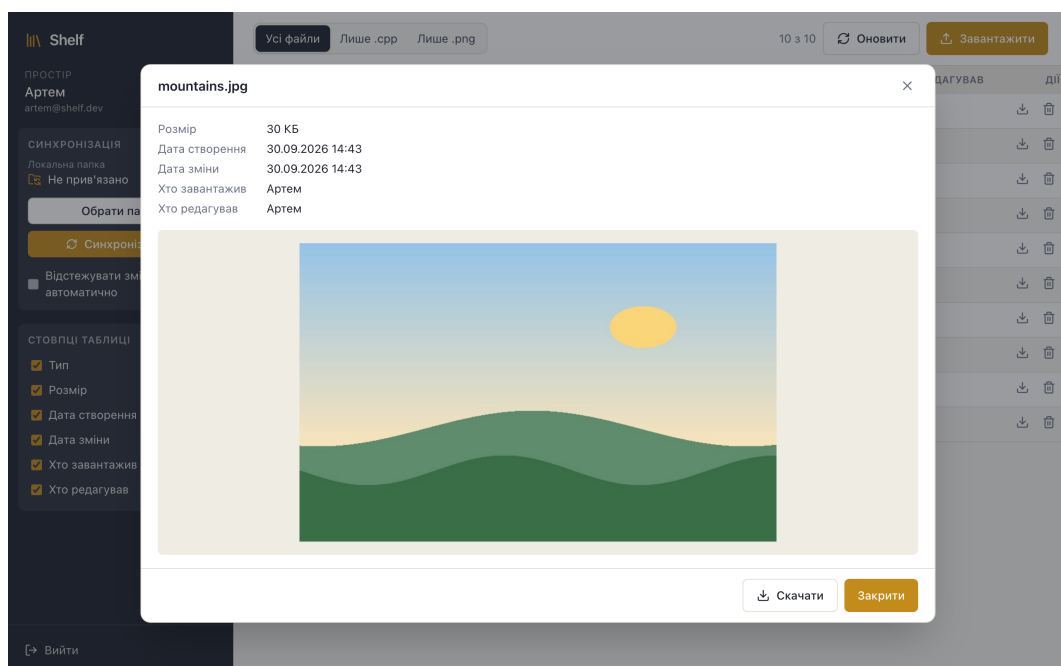


Рисунок 9. Перегляд файлу .jpg як зображення

Для решти типів, як-от архіву `backup.zip` на рис. 10, вікно показує лише атрибути і напис «Перегляд недоступний для цього типу файлу», а вміст такого файлу клієнт навіть не скачує. Закрити вікно можна кнопкою «Закрити», хрестиком, клавішею `Escape` або кліком поза ним. `PreviewDialog` — спільний компонент, тому вікно не відрізняється від десктопного. Наскрізна перевірка відкриває `.kt`, `.jpg` і файл без перегляду й перевіряє кожен варіант вмісту.

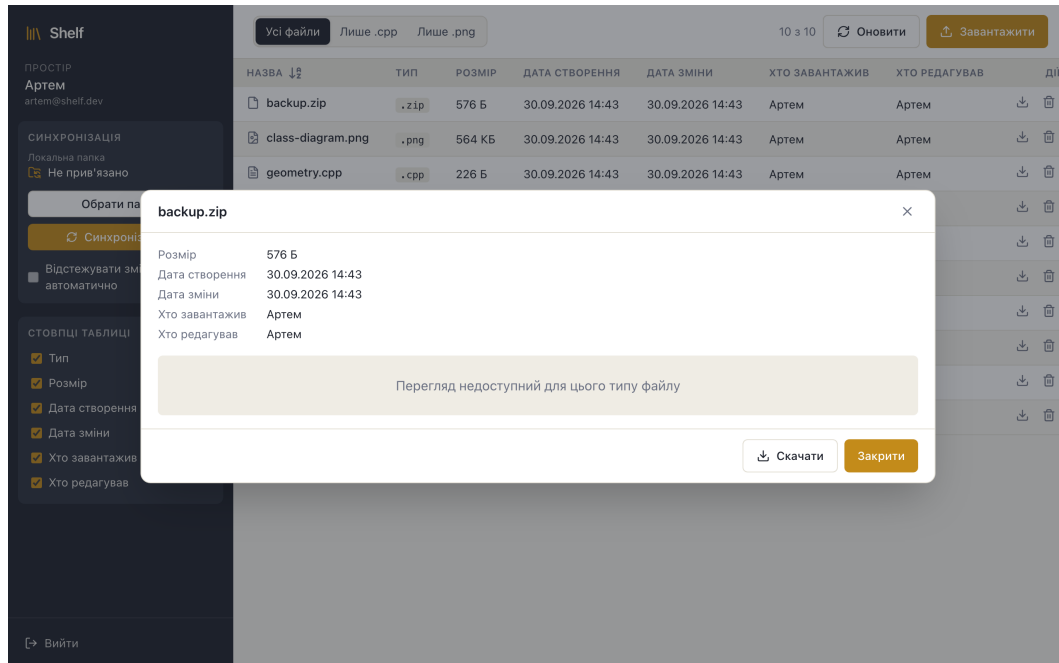


Рисунок 10. Файл без перегляду: лише атрибути

3.7 Завантаження кнопкою і перетягуванням

Вивантаження на сервер покриває вимоги R7 і R9, прецеденти UC9a і UC9b. Кнопка «Завантажити» відкриває діалог браузера з вибором кількох файлів. Бонусний спосіб — перетягнути файли з Finder чи Провідника на таблицю: `UploadDropzone` накриває її шаром із пунктирною рамкою і написом «Відпустіть файли, щоб завантажити їх у простір» (рис. 11). Шар з’являється, лише коли в `dataTransfer.types` є `Files`, тобто тягнуть саме файли, і зникає, коли курсор залишає область.

Після того як файл відпущено (рис. 12), `Greeting.kt` з’явився в таблиці, лічильник показує «11 з 11», а повідомлення — «Завантажено файлів: 1». Ліміт 50 МБ перевіряється двічі: `splitBySize` відкладає завеликі файли ще до запиту й називає їх у повідомленні «Більші за 50 МБ і пропущені: ...», а сервер на такий файл відповідає кодом 413. Наскрізна перевірка кидає разом малий файл і файл понад 50 МБ і переконується, що вивантажено лише малий. Файл з уже наявною назвою стає новою версією (UC10).

3.8 Скачування і видалення файлів

Скачування (UC11) запускає кнопка зі стрілкою в рядку або «Скачати» у вікні перегляду: `FileApiClient.download` отримує вміст із токеном, а `saveBlob` створює для `Blob` тимчасовий object URL, натискає невидиме посилання з атрибутом `download` і через 10 с звільняє URL.

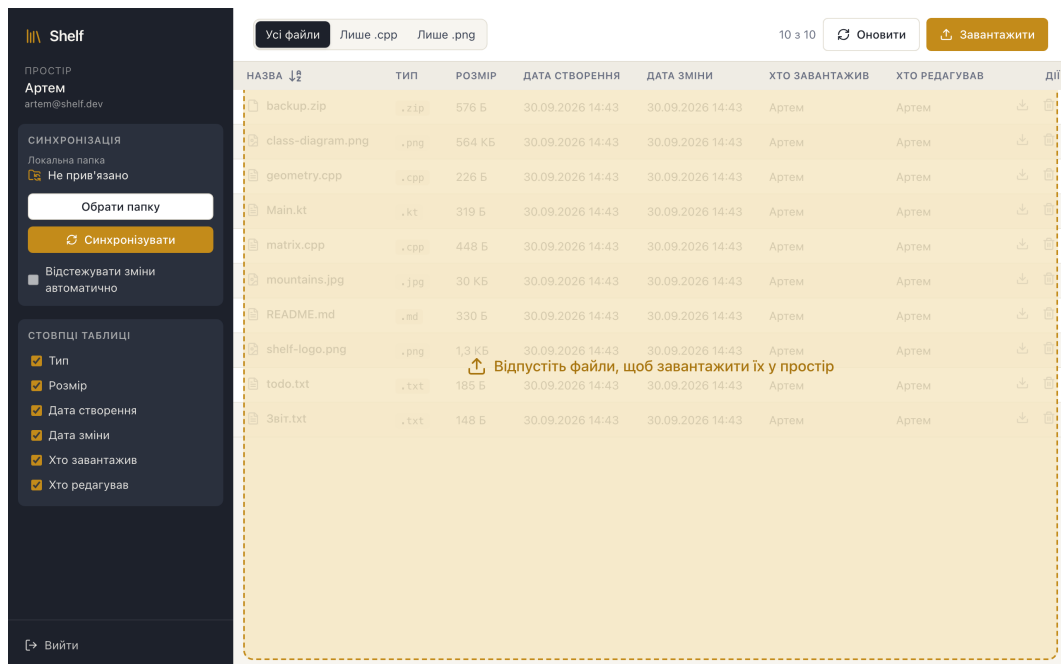


Рисунок 11. Файл перетягують на таблицю

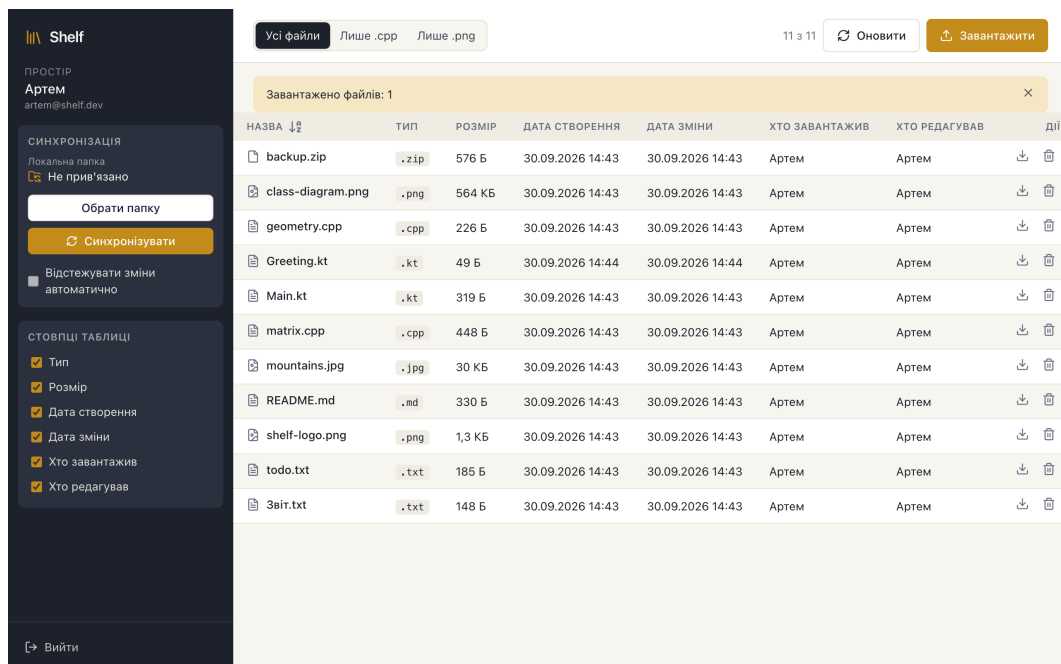


Рисунок 12. Результат перетягування: файл вивантажено

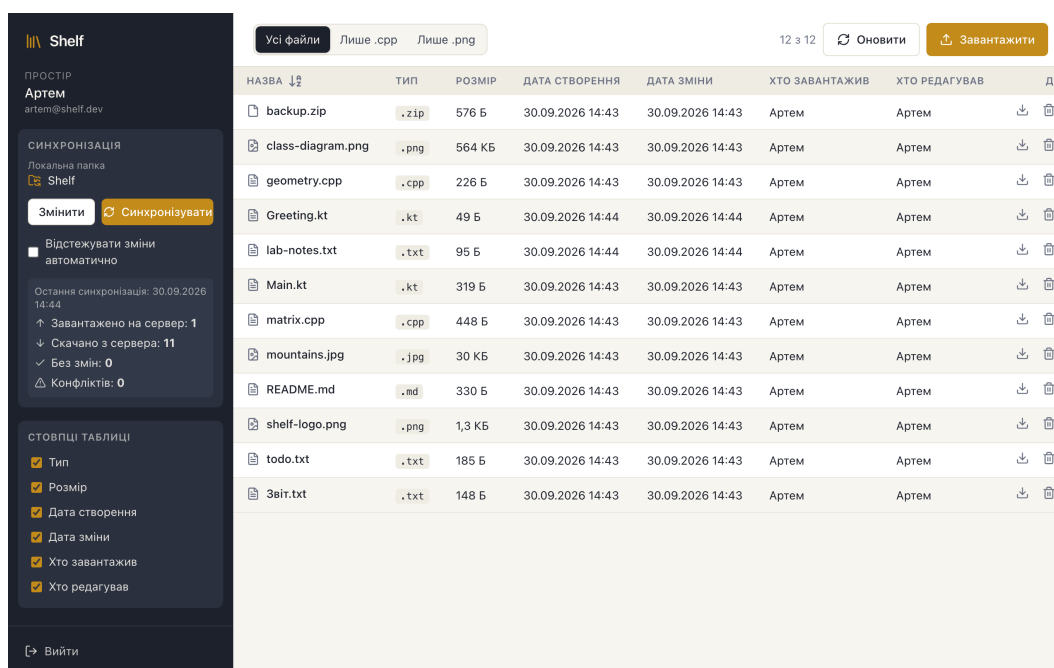
Перетягування рядка з вікна у файловий менеджер (UC11a) лишилося тільки в десктопі: стандартного способу створити файл там, де відпущено кнопку миші, браузер сторінці не дає.

Кнопка з кошиком видаляє файл (UC12). Після стандартного підтвердження браузера «Видалити «назва» з простору?» клієнт надсилає DELETE /api/workspace/files/:id, сервер видаляє запис і об'єкт у сховищі, а таблиця перерахує список. Як і в десктопі, синхронізація видалень не поширює.

3.9 Синхронізація з локальною папкою

Блок «Синхронізація» реалізує вимогу R8 і прецеденти UC13 та UC14. «Обрати папку» викликає `showDirectoryPicker` з доступом на читання і запис; сторінка отримує `handle` папки, але не шлях до неї, тому в панелі видно лише назву. «Синхронізувати» перевіряє дозвіл на папку, після чого `WebSyncService` викликає `scan()` рушія `SyncEngine`, який порівнює папку, список на сервері і знімок з `IndexedDbSnapshotStore`.

На рис. 13 — перша синхронізація папки `Shelf`, де лежав лише `lab-notes.txt`: «Завантажено на сервер: 1», «Скачано з сервера: 11», «Без змін: 0», «Конфліктів: 0», у таблиці 12 файлів. Замість «Обрати папку» з'явилася кнопка «Змінити»: інша папка отримує новий знімок, а повторний вибір тієї самої зберігає старий. Системний діалог вибору папки `Playwright` відкрити не може, тому скрипт зйомки підставляє каталог `Shelf` з приватної файлової системи сайту (`Origin Private File System`), з яким клієнт працює через той самий `File System Access API`.



The screenshot shows the 'Shelf' application interface. On the left, a sidebar contains a 'ПРОСТІР' (Space) section for 'Артем' (Artem) with email 'artem@shelf.dev'. Below this is a 'СИНХРОНІЗАЦІЯ' (Synchronization) section showing 'Локальна папка' (Local folder) as 'Shelf'. It includes buttons for 'Змінити' (Change) and 'Синхронізувати' (Sync). A status bar indicates: 'Відстежувати зміни автоматично' (Track changes automatically), 'Остання синхронізація: 30.09.2026 14:44' (Last sync: 30.09.2026 14:44), 'Завантажено на сервер: 1' (Uploaded to server: 1), 'Скачано з сервера: 11' (Downloaded from server: 11), 'Без змін: 0' (No changes: 0), and 'Конфліктів: 0' (Conflicts: 0). At the bottom of the sidebar is a 'СТОВПЦІ ТАБЛИЦІ' (Table columns) section with checkboxes for 'Тип' (Type), 'Розмір' (Size), 'Дата створення' (Creation date), 'Дата зміни' (Modification date), 'Хто завантажив' (Who uploaded), and 'Хто редагував' (Who edited), all of which are checked. The main area displays a table of files with columns: 'НАЗВА' (Name), 'ТИП' (Type), 'РОЗМІР' (Size), 'ДАТА СТВОРЕННЯ' (Creation date), 'ДАТА ЗМІНИ' (Modification date), 'ХТО ЗАВАНТАЖИВ' (Who uploaded), 'ХТО РЕДАГУВАВ' (Who edited), and 'ДІЇ' (Actions). The table contains 12 rows of files, including 'backup.zip', 'class-diagram.png', 'geometry.cpp', 'Greeting.kt', 'lab-notes.txt', 'Main.kt', 'matrix.cpp', 'mountains.jpg', 'README.md', 'shelf-logo.png', 'todo.txt', and '3bit.txt'. Each row shows the file's details and provides download and delete actions.

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ	ХТО ЗАВАНТАЖИВ	ХТО РЕДАГУВАВ	ДІЇ
backup.zip	.zip	576 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
class-diagram.png	.png	564 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
geometry.cpp	.cpp	226 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
Greeting.kt	.kt	49 Б	30.09.2026 14:44	30.09.2026 14:44	Артем	Артем	↓ 🗑
lab-notes.txt	.txt	95 Б	30.09.2026 14:44	30.09.2026 14:44	Артем	Артем	↓ 🗑
Main.kt	.kt	319 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
matrix.cpp	.cpp	448 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
mountains.jpg	.jpg	30 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
README.md	.md	330 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
shelf-logo.png	.png	1,3 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
todo.txt	.txt	185 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑
3bit.txt	.txt	148 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	↓ 🗑

Рисунок 13. Результат першої синхронізації

3.10 Конфлікт версій і його розв'язання

Файл, змінений після останньої синхронізації і в папці, і на сервері, отримує статус `CONFLICT` за правилами спільної функції `computeStatus`. Для рис. 14 на сервер вивантажено іншу версію `todo.txt` (66 Б), а потім змінено локальний файл (114 Б). «Синхронізувати» відкрило `ConflictDialog` (UC14b), де для кожного файлу вибирають локальну версію або версію з сервера; новішу, тут локальну, вибрано заздалегідь. «Застосувати» передає вибір у `run(resolutions)`, а «Скасувати синхронізацію» відкидає план і нічого не переносить.

Рис. 15 показує результат після «Застосувати»: локальну версію вивантажено, звіт містить «Завантажено на сервер: 1», «Без змін: 11» і «Конфліктів: 1», а `todo.txt` у таблиці має 114 Б і новий час зміни. Поле «Конфліктів» рахує конфлікти, розв'язані в цьому запуску. Файл, змінений, поки діалог був відкритий, під час повторної перевірки плану пропускається з

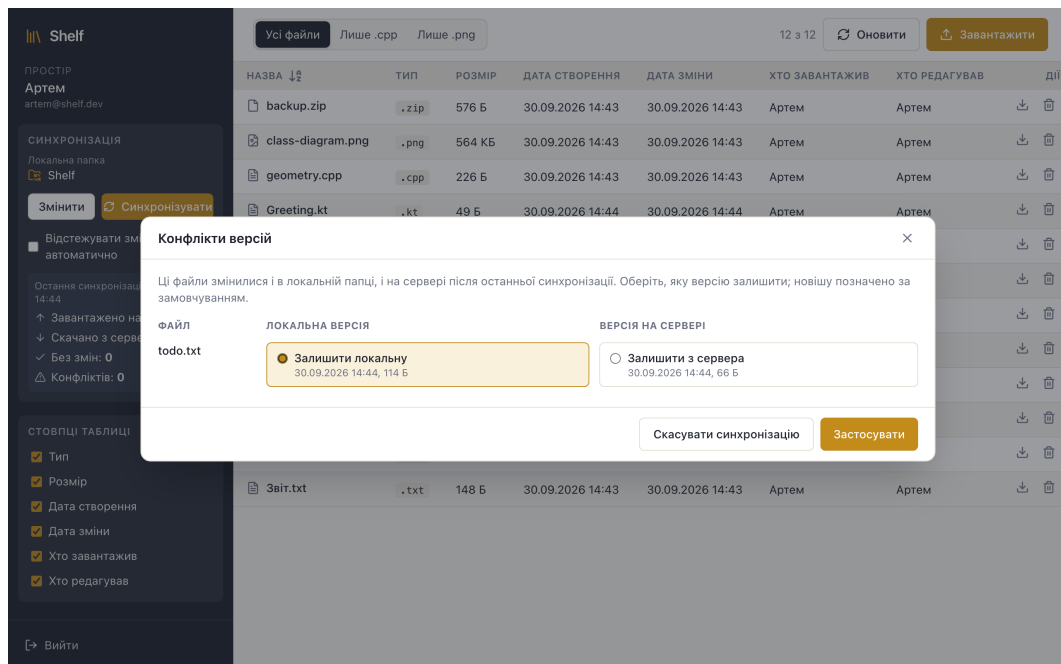


Рисунок 14. Діалог конфліктів версій

повідомленням у звіті, а помилки файлової системи браузера `explainBrowserError` перекладає українською.

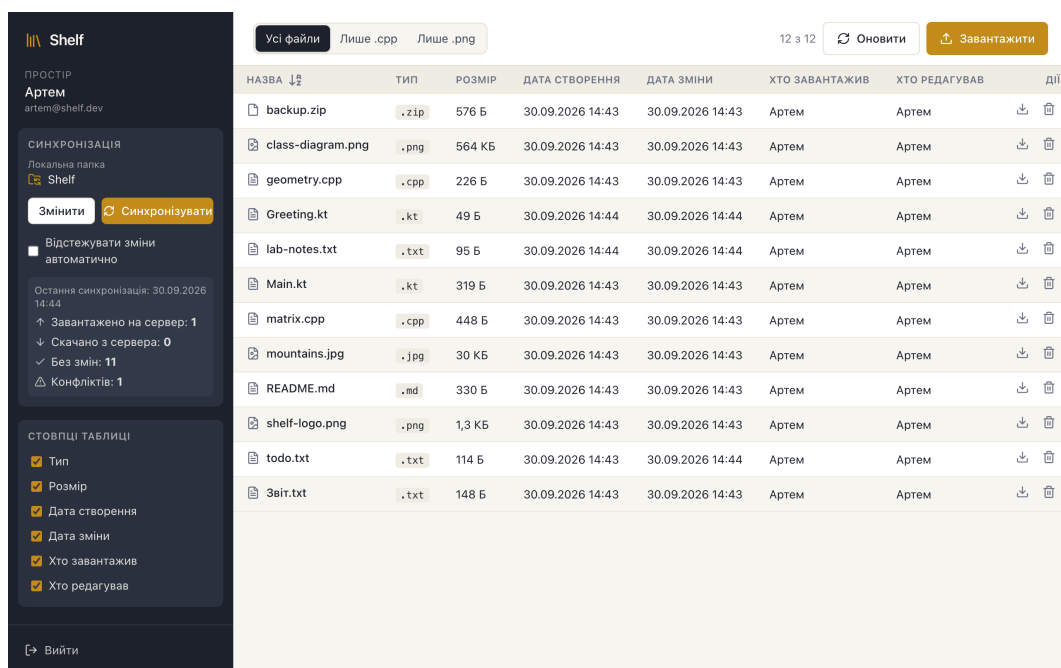
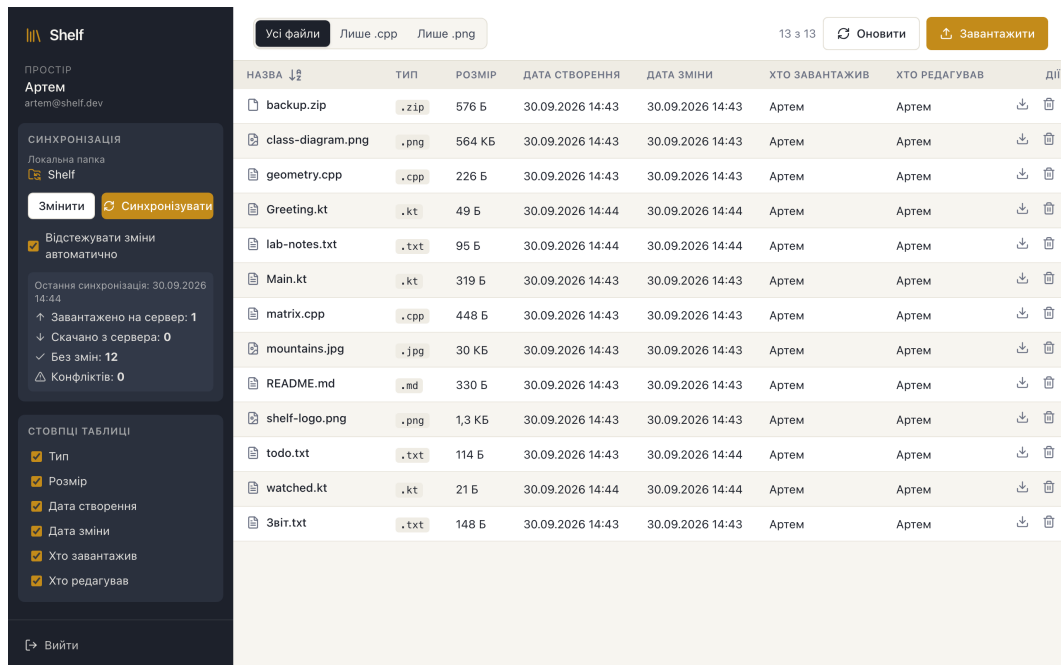


Рисунок 15. Звіт після розв'язання конфлікту

3.11 Автоматичне відстеження змін

Прапорець «Відстежувати зміни автоматично» реалізує прецедент UC14a опитуванням (підрозділ 2.4): коли список файлів папки змінився, `WebSyncService` виконує `synchronize()` без діалогу, і конфлікт розв'язується на користь новішої версії. Як і в десктопному `SyncService`, одночасно працює один запуск, відкритий діалог конфліктів притримує автоматичні запуски, а ручна синхронізація чекає на автоматичну.

На рис. 16 файл `watched.kt`, що з'явився в папці, сам потрапив на сервер: таблиця показує «13 з 13», а звіт — «Завантажено на сервер: 1» і «Без змін: 12». Прапорець зберігається разом із прив'язкою папки облікового запису. Після виходу опитування зупиняється, а результат запуску, що завершився пізніше, відкидається. Якщо після перезавантаження сторінки дозволу на папку немає, блок один раз просить натиснути «Синхронізувати» і дозволити доступ, а опитування чекає.



The screenshot shows the Shelf application interface. On the left, a sidebar displays the user's profile (Artem, artem@shelf.dev) and synchronization status. The main area shows a table of files with columns for name, type, size, creation date, modification date, uploader, and editor. The file `watched.kt` is highlighted in the table.

НАЗВА	ТИП	РОЗМІР	ДАТА СТВОРЕННЯ	ДАТА ЗМІНИ	ХТО ЗАВАНТАЖИВ	ХТО РЕДАГУВАВ	ДІЇ
backup.zip	.zip	576 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
class-diagram.png	.png	564 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
geometry.cpp	.cpp	226 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
Greeting.kt	.kt	49 Б	30.09.2026 14:44	30.09.2026 14:44	Артем	Артем	📄 🗑️
lab-notes.txt	.txt	95 Б	30.09.2026 14:44	30.09.2026 14:44	Артем	Артем	📄 🗑️
Main.kt	.kt	319 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
matrix.cpp	.cpp	448 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
mountains.jpg	.jpg	30 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
README.md	.md	330 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
shelf-logo.png	.png	1,3 КБ	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️
todo.txt	.txt	114 Б	30.09.2026 14:43	30.09.2026 14:44	Артем	Артем	📄 🗑️
watched.kt	.kt	21 Б	30.09.2026 14:44	30.09.2026 14:44	Артем	Артем	📄 🗑️
3bit.txt	.txt	148 Б	30.09.2026 14:43	30.09.2026 14:43	Артем	Артем	📄 🗑️

Рисунок 16. Автоматична синхронізація після появи файлу в папці

3.12 Вузький екран

Коли вікно вужче за 768 px, `WorkspaceLayout` ховає бічну панель: у панелі інструментів з'являється кнопка меню, кнопки переносяться на другий рядок, а таблицю можна прокручувати вбік (рис. 17, ліворуч). Кнопка меню висуває панель поверх вмісту (праворуч) з тими самими блоками синхронізації і стовпців; закривають її хрестиком або кліком по затемненій частині сторінки. Знімки зроблено у вікні 390×844, як на смартфоні. Синхронізація на телефоні залежить від браузера: там, де немає `showDirectoryPicker` (зокрема в Safari), блок покаже пояснення з наступного підрозділу.

3.13 Браузери без File System Access API

`showDirectoryPicker` є лише в браузерах на основі Chromium, зокрема Chrome і Edge, і тільки на сторінках, відкритих через HTTPS або з `localhost`. Якщо функція `directoryPicker` його не знаходить, `useWebSync` не створює `WebSyncService`, а `SyncPanel` показує замість кнопок пояснення «Синхронізація з локальною папкою працює в Chrome і Edge (потрібен File System Access API)...» (рис. 18); решта функцій працює як звичайно. Для знімка в Chromium прибрано `showDirectoryPicker`, щоб він поведився як Firefox чи Safari; цей випадок перевіряє і наскрізний тест.

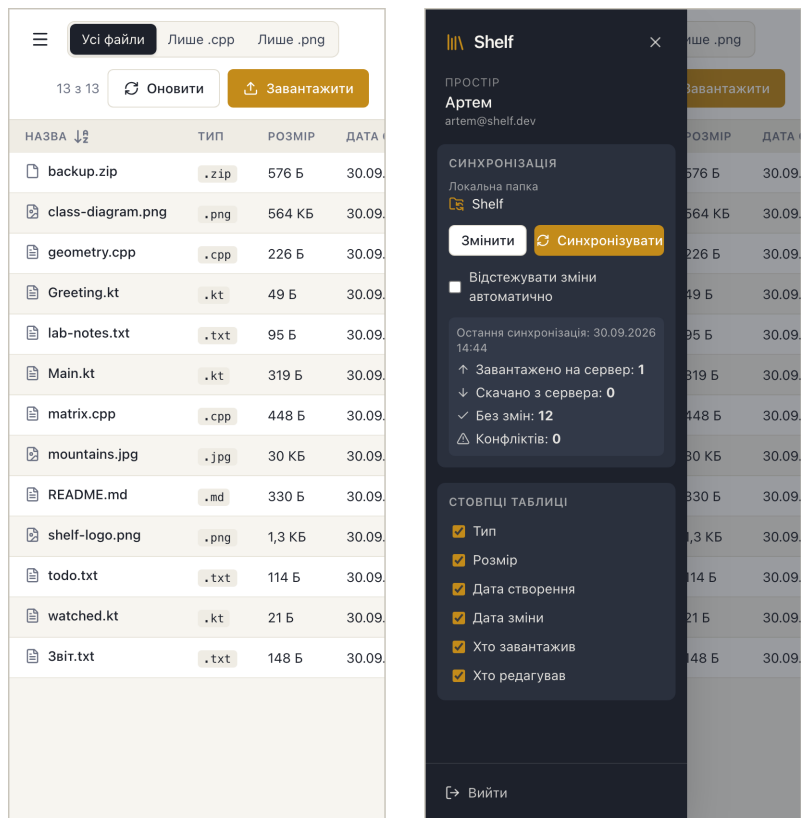


Рисунок 17. Вузьке вікно 390×844: таблиця (ліворуч) і відкрита бічна панель (праворуч)

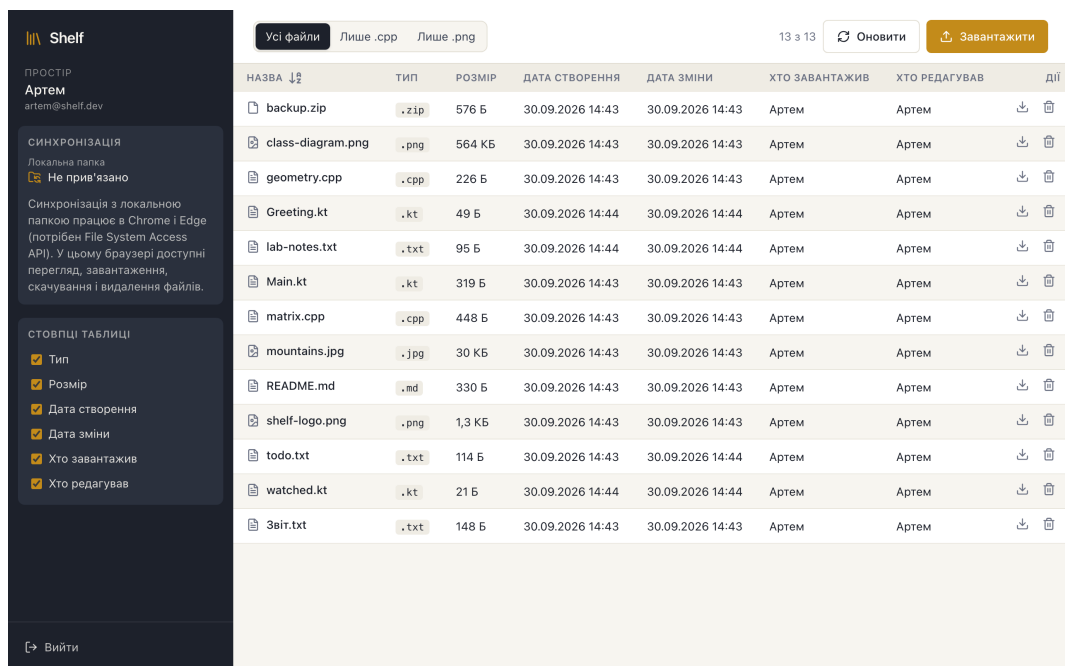


Рисунок 18. Браузер без File System Access API: пояснення замість кнопок синхронізації

4 Публікація

4.1 Схема розгортання

Опублікована система відповідає діаграмі розгортання етапу 1 (рис. 19). Браузер отримує сторінки з Vercel, а дані — від API на Render; сервер у контейнері Docker звертається до Neon через Prisma і до Supabase Storage через S3 API; десктоп працює з тим самим API, а локальний

Docker Compose запускає той самий образ із PostgreSQL і MinIO. Уточнень два: сервер слухає порт зі змінної PORT, яку задає Render (локально — 4000), а не 3000, як на діаграмі; сторінки Next.js статичні, серверного рендерингу немає.

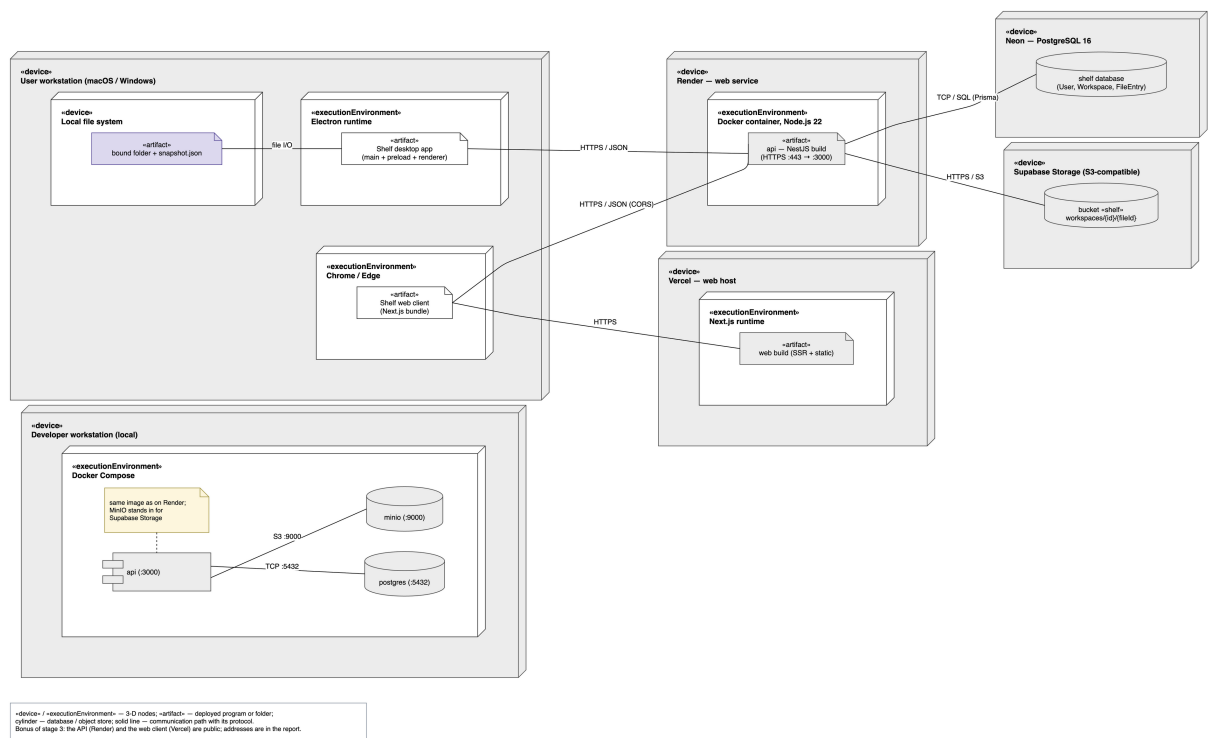


Рисунок 19. Діаграма розгортання з етапу 1

Компонент	Сервіс	План	Адреса або роль
веб-клієнт	Vercel	Hobby	shelf-opal-two.vercel.app, статичні сторінки Next.js
REST API	Render, Docker web service з render.yaml	Free, Frankfurt	shelf-api-1989.onrender.com, NestJS у контейнері
база даних	Neon PostgreSQL, пряме з'єднання	Free, Frankfurt	користувачі, простори, записи про файли
сховище файлів	Supabase Storage, S3-сумісний ендпоінт	Free	вміст файлів у бакеті shelf
CI	GitHub Actions	безкоштовний	збірка і тести на кожен push і pull request

4.2 Опублікована система

Рис. 20 — опублікований сайт <https://shelf-opal-two.vercel.app> після входу як artem@shelf.dev: ті самі десять демонстраційних файлів, що й локально, з датою створення 30.09.2026 14:08, коли їх вивантажено на опублікований сервер (підрозділ 4.5). Сторінку віддав Vercel, список файлів прийшов від API на Render, а той прочитав метадані з Neon. Під час зйомки в опублікованому просторі нічого не змінювалося, а наскрізна перевірка з розділу 5 працює з тимчасовими обліковими записами.

На рис. 21 — Swagger UI опублікованого сервера (<https://shelf-api-1989.onrender.com/api/docs>). Маршрути згруповано за контролерами health, auth, workspace і files; замок позначає маршрути, що потребують токена, а кнопка «Authorize» дає змогу вставити JWT і викликати їх

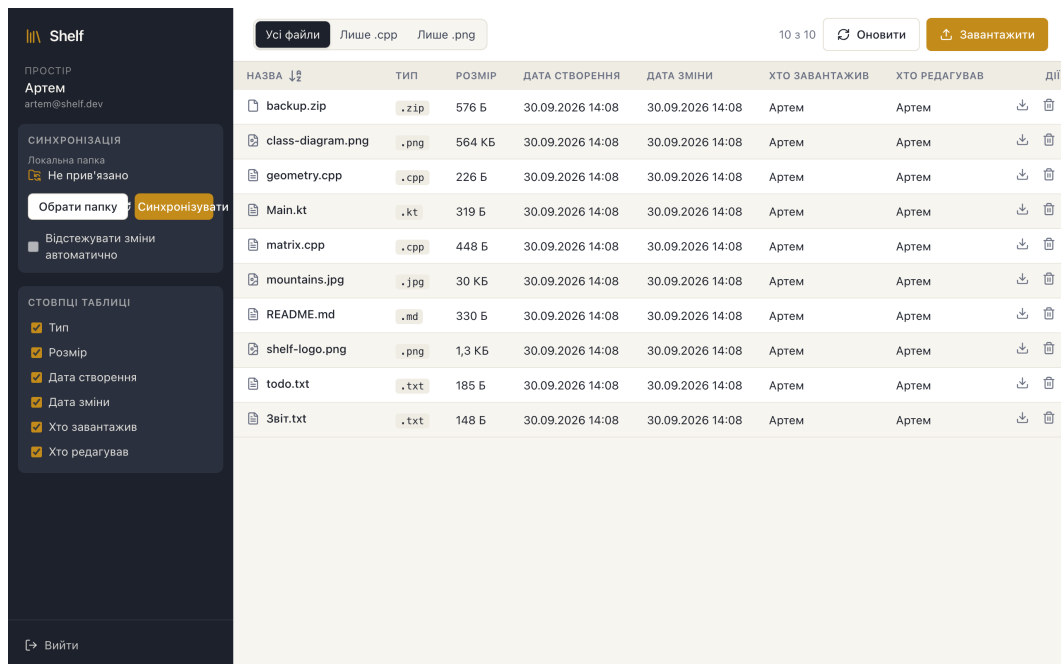


Рисунок 20. Опублікований веб-клієнт: простір демонстраційного користувача

просто з браузера. Сторінка підтверджує, що за публічною адресою працює той самий API, що й локально, і дозволяє перевірити реєстрацію, вхід, вивантаження та скачування без клієнта. Разом із сайтом це становить бонус за публікацію.

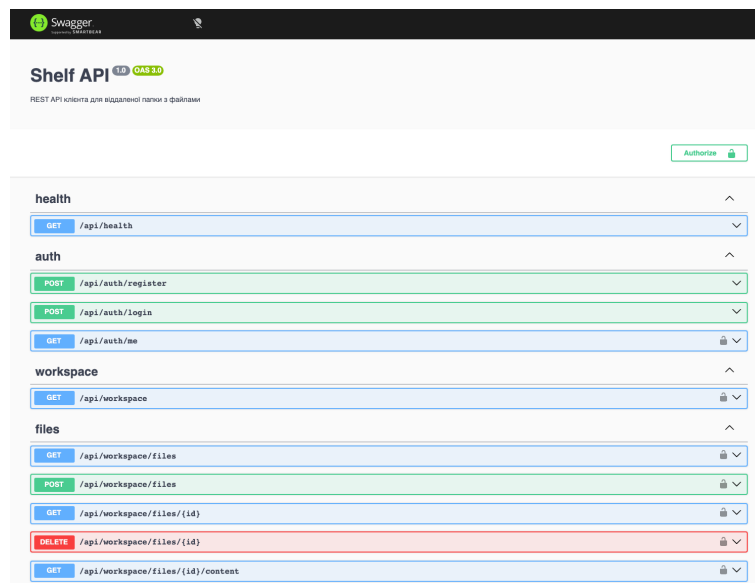


Рисунок 21. Swagger UI опублікованого сервера

4.3 Сервер на Render

Сервер описано у файлі `render.yaml` (Render Blueprint): сервіс `shelf-api` типу `web`, середовище `Docker`, план `free`, регіон `frankfurt`. Образ збирається з того самого `docker/api.Dockerfile`, що й локально: перший етап встановлює лише `@shelf/api` із залежностями і збирає його разом з `@shelf/shared`, другий містить тільки готові файли й працює від користувача `node`. Контейнер спершу виконує `prisma migrate deploy`, а потім запускає сервер; готовність Render перевіряє запитом `/api/health`.

Секретів у репозиторії немає: змінні з позначкою `sync: false` (`DATABASE_URL`, адресу, регіон і ключі `S3`, `CORS_ORIGINS`) введено в панелі `Render` під час створення сервісу, `JWT_SECRET` `Render` згенерував сам (`generateValue: true`), а у файлі записано лише бакет `shelf`, адресу S3 за шляхом і ввімкнений `Swagger`. `DATABASE_URL` — пряме з'єднання з Neon (теж Frankfurt) без пулера, бо ту саму змінну використовує `prisma migrate deploy`, якому потрібен сеанс із базою напряду. `Supabase Storage` підключено через S3-сумісний ендпоінт; `StorageService` працює з ним тим самим кодом, що й з `MinIO`.

4.4 Веб-клієнт на Vercel

Проект `Vercel` під'єднано до репозиторію `GitHub` з коренем `apps/web` (`Root Directory`). Оскільки це монорепозіторій `rnrpm`, команди встановлення і збірки задано у файлі `apps/web/vercel.json`:

```
ELECTRON_SKIP_BINARY_DOWNLOAD=1 pnpm install --filter @shelf/web...  
pnpm --filter @shelf/web... run build
```

Перша команда встановлює лише веб-клієнт разом із `@shelf/shared` і `@shelf/ui` та не скачує `Electron`, потрібний тільки десктопу. Друга спершу збирає `@shelf/shared`, а потім сам клієнт. Адресу API задає змінна проекту `NEXT_PUBLIC_API_URL`, яку `Next.js` вбудовує в код під час збірки; якщо її не задано, перевірка в `next.config.ts` зупиняє збірку на `Vercel`, і сайт, що звертався б до `http://localhost:4000`, не публікується.

4.5 CORS і демонстраційні дані

Сторінки й API мають різні домени, тож відповіді API сторінка читає лише з дозволу `CORS`. Сервер дозволяє будь-яке джерело (*): `NestJS` повторює в `Access-Control-Allow-Origin` адресу сторінки, що надіслала запит. Це безпечно, бо сесія передається не в `cookie`, а токеном у заголовку `Authorization`, який браузер сам не додає: чужа сторінка без токена отримає 401, а токен з `localStorage` сайту `Shelf` прочитати не зможе. Перелік дозволених адрес довелося б доповнювати ще й сторінкою десктопа, відкритою з локального файлу.

Демонстраційні дані записано тією самою командою `pnpm seed`, що й локально, лише зі змінною `SHELF_API`, яка містить адресу опублікованого API. Скрипт `tools/seed-demo.mjs` не звертається до бази напряду, а реєструє користувачів (або входить, якщо вони вже є) і вивантажує файли через публічний `REST API`, тож його можна запускати повторно.

4.6 Обмеження безкоштовних планів

Для демонстрації важливі два обмеження. Безкоштовний сервіс `Render` засинає після 15 хвилин без запитів, і перший запит чекає до хвилини, поки запусниться контейнер (клієнт через 5 с показує про це підказку), тож сайт краще відкрити заздалегідь. `Supabase` призупиняє безкоштовний проект після 7 днів без активності, і вміст файлів стає недоступним, доки проект не відновлять у його панелі, тож напередодні захисту сховище варто перевірити.

5 Тестування

5.1 Unit-тести

`pnpm test` у корені запускає тести всіх пакетів: їх 300, і всі проходять (GitHub Actions виконує їх на кожен `push` і `pull request`). Порівняно з етапом 2 додано тести `accountKey`, `@shelf/ui` і весь набір `@shelf/web`.

Набір	Засіб	Що перевіряє	Тестів
@shelf/shared	Vitest, 12 файлів	операції варіанта, перегляд, <code>FileApiClient</code> , <code>SyncEngine</code>	142
@shelf/ui	Vitest, 1 файл	текст повідомлення після завантаження	2
@shelf/api	Jest, 8 наборів	<code>AuthService</code> , <code>FilesService</code> , <code>StorageService</code> , помилки	58
@shelf/desktop	Vitest, 7 файлів	<code>NodeLocalFolder</code> , <code>FolderWatcher</code> , <code>SyncService</code>	42
@shelf/web	Vitest, 7 файлів	сесія, <code>BrowserLocalFolder</code> , <code>IndexedDB</code> , <code>WebSyncService</code>	56
разом			300

Тести веб-клієнта працюють у Node без браузера: `File System Access API` замінюють `handle-и` в пам'яті з поведінкою Chromium (`src/test/fakeFileSystem.ts`), `IndexedDB` — `fake-indexeddb`, сервер — `API` в пам'яті. Вони перевіряють ризики, властиві саме браузеру: скачаний файл наступного разу має статус `IN_SYNC` і не вивантажується назад; файли `.crgswar` не потрапляють до списку; без дозволу на папку, як після перезавантаження, відстеження чекає, а ручна синхронізація запитує дозвіл; другий обліковий запис не отримує папки першого; невдале скачування нового файлу не лишає порожнього файлу, який наступний запуск вивантажив би на сервер.

Тіло першого з цих тестів (`browserLocalFolder.test.ts`): через годину після скачування сканування бачить `IN_SYNC`, а вивантажень на сервер немає.

```
const api = new FakeApi();
const snapshots = new MemorySnapshotStore();
const engine = () => new SyncEngine({ localFolder: folder, snapshotStore: snapshots, api });
const entry = await api.put('photo.jpg', 'jpg-bytes');

expect(await engine().synchronize()).toMatchObject({ downloaded: 1, uploaded: 0, errors: [] });
expect(root.lastModified('photo.jpg')).not.toBe(Date.parse(entry.modifiedAt));

clock.advance(HOUR);
expect((await engine().scan()).map((item) => item.status)).toEqual([SyncStatus.IN_SYNC]);
expect(await engine().synchronize()).toMatchObject({ uploaded: 0, downloaded: 0, skipped: 1 });
expect(api.uploads).toBe(0);
```

5.2 Наскрізна перевірка

Сценарій `apps/web/scripts/smoke.mjs` (`pnpm -F @shelf/web smoke`) збирає клієнт, запускає `next start`, створює новий обліковий запис і через Playwright керує Chromium. Його 20 кроків: перехід на вхід, коли сервер відхиляє збережений токен, вхід і сортування, спадання з «Лише .crr», «Лише .png», стовпці, три види перегляду, зона перетягування без файлів і з файлами, перетягування з пропуском файлу понад 50 МБ, кнопка «Завантажити» і видалення, скачування, перша синхронізація без `.crgswar`, повторна без змін, конфлікт, відстеження, вузьке вікно, вихід, реєстрація, папка облікового запису і браузер без `File System Access API`.

Той самий сценарій зі змінними SHELF_WEB і SHELF_API пройшов усі 20 кроків на опублікованих сайті й сервері, а tools/api-smoke.sh виконав 20 перевірок опублікованого API (реєстрація, коди 401, 409, 413 і 404, нова версія файлу, видалення); обидві перевірки створюють власні тимчасові облікові записи (фрагмент журналів; повністю — docs/reports/img/stage3/smoke-api-prod.txt і smoke-web-prod.txt):

```
$ SHELF_API=https://shelf-api-l989.onrender.com bash tools/api-smoke.sh
ok re-upload same name → 200
ok 51 MB → 413
api-smoke: OK
$ SHELF_WEB=https://shelf-opal-two.vercel.app \
  SHELF_API=https://shelf-api-l989.onrender.com node apps/web/scripts/smoke.mjs
ok login, table sorted ascending
ok a second sync changes nothing: downloaded files are not uploaded back
ok conflict dialog: the chosen server version replaced the local file
web-smoke: OK
```

6 Інструкція із запуску

Потрібні Node.js 22.12 або новіший, pnpm 10 (corepack enable pnpm) і Docker. Локальний запуск:

```
pnpm install
pnpm build # збирає пакети, серед них @shelf/shared
cp docker/.env.example docker/.env
pnpm stack:up # PostgreSQL, MinIO і API на :4000
pnpm seed # демонстраційні користувачі та файли
pnpm dev:web # веб-клієнт на http://localhost:3000
```

Синхронізація з папкою працює в Chrome або Edge, і для localhost HTTPS не потрібен. Опубліковану систему можна відкрити без встановлення: сайт <https://shelf-opal-two.vercel.app>, документація API <https://shelf-api-l989.onrender.com/api/docs>. Демонстраційні облікові записи: artem@shelf.dev (Артем, десять файлів), iryna@shelf.dev (Ірина, три файли) і maksym@shelf.dev (Максим, два файли); пароль у всіх shelf-demo-2026. Після 15 хвилин без запитів сервер засинає, тож перший вхід може тривати до хвилини.

7 Висновки

На етапі 3 створено веб-версію Shelf на Next.js 16. Вона виконує вимоги R1–R8, зокрема сортування за назвою, фільтр «усі / лише .cpp / лише .png», перегляд .kt як тексту і .jpg як зображення, а також бонусне завантаження перетягуванням. Сервер, база, сховище й веб-клієнт опубліковані на Render, Neon, Supabase і Vercel, і наскрізна перевірка пройшла за публічними адресами. З десктопом спільні операції варіанта, REST-клієнт, рушій синхронізації і всі компоненти інтерфейсу; новими стали сесія, збереження файлів і реалізації LocalFolder та SnapshotStore. Браузер змусив змінити кілька рішень: відстеження змін опитуванням, скачування через Blob, запис у знімок справжнього часу зміни, повторний запит дозволу на папку і синхронізацію лише в Chrome та Edge. Код перевіряють 300 unit-тестів. На етапі 4 ці відмінності двох клієнтів буде порівняно докладно.